

Uniwersytet WSB Merito Chorzów

Kierunek: Informatyka

Rok akademicki 2025/2026

I rok, semestr II

Projekt z Programowania Obiektowego

Temat projektu: Hardware Usage Monitor

Aplikacja do monitorowania zasobów sprzętowych
i zarządzania kontenerami Docker.

Igor Dybał, nr indeksu 183593

Łukasz Frydrych, nr indeksu 183596

Chorzów, 2026

Spis treści

1.	Opis i założenia projektu	Error! Bookmark not defined.
2.	Opis techniczny projektu	Error! Bookmark not defined.
2.1.	Interfejsy.....	Error! Bookmark not defined.
2.2.	Klasy.....	Error! Bookmark not defined.
2.3.	Klasy.....	Error! Bookmark not defined.
2.4.	Hermetyzacja.....	Error! Bookmark not defined.
2.5.	Dziedziczenie	Error! Bookmark not defined.
2.6.	Polimorfizm	Error! Bookmark not defined.
3.	Wnioski	Error! Bookmark not defined.

1. Opis i założenia projektu

Celem projektu było stworzenie aplikacji Hardware Usage Monitor, służącej do bieżącego monitorowania zasobów systemowych oraz wyświetlania informacji o uruchomionych kontenerach Docker. Projekt umożliwił zastosowanie w praktyce zagadnień związanych z programowaniem obiektowym w języku C#, takich jak podział odpowiedzialności pomiędzy klasy, enkapsulacja oraz organizacja kodu aplikacji.

Założenia projektu

- **Interfejs graficzny** - zamiast aplikacji konsolowej został stworzony prosty interfejs graficzny z wykorzystaniem Avalonia UI. Dzięki temu aplikacja może być rozwijana nie tylko dla systemu Windows, ale także dla systemów z rodziny Linux oraz macOS.
- **Architektura MVVM (Model-View-ViewModel)** - kod został zorganizowany zgodnie ze wzorcem MVVM, aby oddzielić logikę pobierania i przetwarzania danych (modele i serwisy) od widoków (Views).
- **Monitorowanie zasobów w czasie rzeczywistym** - wykorzystanie natywnych bibliotek systemowych pozwoliło na odczytywanie zużycia pamięci RAM oraz obciążenia procesora z podziałem na rdzenie i wątki.
- **Integracja z systemem Docker** - program parsuje dane z wiersza poleceń, agreguje kontenery w obrębie projektów Docker Compose i umożliwia podstawowe zarządzanie nimi, czyli uruchamianie oraz zatrzymywanie z poziomu aplikacji.
- **Cykliczne odświeżanie danych** - zastosowano asynchroniczne odświeżanie danych z użyciem mechanizmów DispatcherTimer oraz Task. Pozwala to na ciągłe wyświetlanie aktualnych informacji przy zachowaniu płynności interfejsu użytkownika.

2. Opis techniczny projektu

Architektura aplikacji opiera się na zasadach programowania obiektowego, zapewniając łatwość rozbudowy i utrzymania kodu. Struktura projektu obejmuje następujące kluczowe elementy:

2.1. Interfejsy

- **2.1.1. ISystemMonitorService** - określa metody do pobierania statystyk systemu, takich jak obciążenie procesora, dane dotyczące rdzeni CPU, zużycie pamięci RAM oraz lista procesów zwracana przez metodę `GetProcesses()`.
- **2.1.2. IDockerService** - określa metody wykorzystywane do operacji związanych z obsługą Dockera, w tym pobierania informacji o kontenerach oraz wykonywania podstawowych działań zarządzających.

2.2. Klasy abstrakcyjne

- **2.2.1. ViewModelBase** - klasa bazowa dla wszystkich modeli widoków (ViewModels). Dziedziczy po `ObservableObject` z biblioteki `CommunityToolkit.Mvvm`, co zapewnia implementację mechanizmu `INotifyPropertyChanged` i umożliwia automatyczne odświeżanie danych w interfejsie użytkownika.
- **2.2.2. SystemMonitorServiceBase** - zapewnia wspólną logikę oraz mechanizmy wykorzystywane przez różne implementacje monitorowania sprzętu.

2.3. Klasy finalne

- **2.3.1. WindowsSystemMonitorService** - implementacja klasy `SystemMonitorServiceBase` przeznaczona dla systemu Windows.
- **2.3.2. DockerCliService** - implementacja wywołująca polecenia Dockera z poziomu aplikacji i przetwarzająca zwrócone dane.

2.4. Hermetyzacja

Hermetyzacja została zastosowana poprzez ograniczenie bezpośredniego dostępu do wewnętrznego stanu obiektów. Klasy z grupy `ViewModel`, na przykład `MainWindowViewModel`, modyfikują dane za pomocą prywatnych setterów oraz metod wewnętrznych, a na zewnątrz udostępniają głównie informacje przeznaczone do odczytu.

2.5. Dziedziczenie

Modele widoków dziedziczą wspólne cechy z klasy bazowej `ViewModelBase`. W podobny sposób zorganizowana została klasa `WindowsSystemMonitorService`, która dziedziczy po `SystemMonitorServiceBase` i rozszerza ją o implementację przeznaczoną dla systemu Windows.

2.6. Polimorfizm

Klasy warstwy ViewModel komunikują się z serwisami na podstawie interfejsów, a nie konkretnych implementacji. Takie rozwiązanie ułatwia testowanie oraz pozwala w przyszłości dodać obsługę innych systemów operacyjnych bez konieczności przebudowy głównej logiki aplikacji.

3. Wnioski

Realizacja projektu umożliwiła praktyczne wykorzystanie zagadnień związanych z programowaniem obiektowym podczas tworzenia rzeczywistej aplikacji desktopowej. Zastosowanie wzorca MVVM pozwoliło na wyraźne oddzielenie warstwy prezentacji od logiki przetwarzania danych, co przełożyło się na większą przejrzystość oraz łatwiejsze utrzymanie kodu.

Wykorzystanie interfejsów i klas abstrakcyjnych sprawiło, że aplikacja została zaprojektowana w sposób elastyczny i przygotowany do dalszego rozwoju. Obecnie obsługuje ona system Windows, jednak zastosowana architektura umożliwia stosunkowo proste rozszerzenie jej działania na inne systemy operacyjne, takie jak Linux czy macOS.

Projekt pozwolił również zdobyć praktyczne doświadczenie w pracy z technologiami wykorzystywanymi we współczesnym środowisku programistycznym. Integracja z Dockerem, monitorowanie zasobów systemowych oraz zastosowanie mechanizmów asynchronicznych pokazały, jak połączyć różne obszary programowania w ramach jednego rozwiązania.

W przyszłości aplikacja mogłaby zostać rozbudowana o dodatkowe funkcjonalności, takie jak prezentacja historycznych danych w postaci wykresów, bardziej zaawansowane zarządzanie kontenerami Docker, obsługa kolejnych platform systemowych oraz możliwość konfiguracji parametrów monitorowania przez użytkownika.